

FORGE — FDE ACADEMY · RAG FOUNDATIONS ON AZURE · Day 6 · 30-Jun-2026

Chunking Strategies for RAG

Choosing the split that wins on recall@k, MRR@k and nDCG@k

Best practices · industry use cases · honest pros & cons · how each split moves retrieval eval

HOW WE'LL SPEND THIS SESSION

What we'll cover



Why chunking decides RAG quality

Context windows, retrieval efficiency, relevance — and the context-loss trap

01



Where the split lives in the pipeline

Index → Retrieve → Augment → Generate, and the metrics that grade it

02



Six strategies, honestly compared

Fixed, semantic, recursive, adaptive, context-enriched, AI-driven

03



Industry use cases

BFSI (Britannia), legal, healthcare, code, support — mapped to strategies

04



Best-practice knobs & advanced trends

Size/overlap, hybrid, metadata, hierarchical & neural chunking

05



Measuring it

Building a gold set and reading recall@k, MRR@k, nDCG@k

06

THE PROBLEM CHUNKING SOLVES

Why chunking decides RAG quality



Context-window limits

Embedding models and LLMs cap how much text they can ingest at once. Right-sized chunks stay under those limits.



Retrieval efficiency

Smaller, precise chunks mean faster lookups and better recall — the right passage is easier to surface.



Computational cost

Appropriately sized chunks avoid wasted tokens and embedding overhead.



Semantic integrity

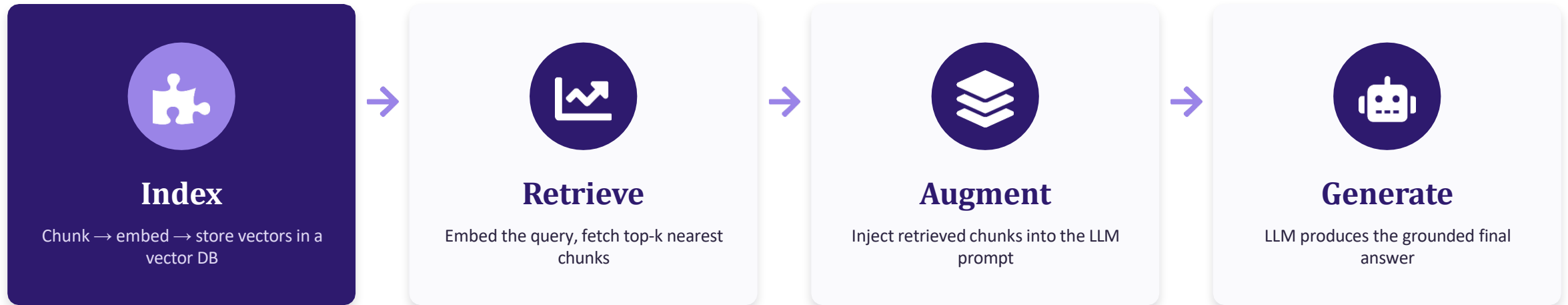
Keeping related ideas together produces cleaner embeddings and more accurate matches.



Continuity — from our naive-RAG session: our answer failed because one client's project sat in chunk 1 while its name sat in chunk 31. Hard splits scatter meaning across chunks and the LLM loses the link. That failure is exactly what a better chunking strategy is meant to prevent.

CHUNKING IS A PREPROCESSING STEP IN INDEXING

Where the split lives in the RAG pipeline



 **Chunking happens once, in the indexing/preprocessing stage — but its quality flows straight into retrieval.** A bad split you make here caps every score you can ever get downstream. You cannot out-prompt a chunk that never gets retrieved.

HOW WE GRADE A CHUNKING STRATEGY

The three-retrieval metrics @k

recall@k

COVERAGE

Of all chunks that are truly relevant, what fraction show up in the top-k retrieved.

Did we surface the right chunk at all?

```
recall@k = (relevant chunks in top-k) / (all relevant chunks)
```

MRR@k

FIRST-HIT RANK

Mean of 1/rank of the first relevant chunk across queries (0 if none in top-k).

How high is the first correct chunk?

```
MRR@k = mean( 1 / rank_of_first_relevant )
```

nDCG@k

RANKING QUALITY

Rewards relevant chunks near the top with graded relevance, discounted by position; normalised to the ideal order.

Is the whole ranking well-ordered?

```
nDCG@k = DCG@k / IDCG@k
```

THE MECHANISM, NOT MAGIC

From split to score: how chunking moves the numbers

TECHADEMY



Split quality

Where you cut decides what each chunk means



Embedding fidelity

A whole idea → a clean vector; a cut idea → a muddy one



Match strength

Cleaner vectors sit closer to the query vector



The @k scores

Right chunk retrieved (recall↑), ranked high (MRR↑, nDCG↑)

✗ Chunks too large

- Multiple topics per chunk dilute the embedding — topic drift
- Query matches weakly → lower recall@k and precision

✗ Chunks too small

- Answer spans several chunks — you need a bigger k to catch it all
- recall@k at small k suffers; context-enriching helps

STRATEGY 1 · THE BASELINE

Fixed-Size Chunking

WHAT IT IS

Split text into equally sized pieces by character, token or word count — usually with an overlap so ideas carry across the cut.

HOW (LangChain)

```
from langchain_text_splitters import CharacterTextSplitter
CharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```



Best fit: uniform, consistently formatted docs — logs, simple text, FAQs.



Pros

- Simplest to implement; uniform sizes ease batching
- Predictable cost; good baseline to measure against



Cons

- Cuts sentences/paragraphs abruptly, ignoring meaning
- Relevant info scatters across chunks (the chunk-1/31 trap)



METRIC LEAN

Middle-of-road recall@k; MRR@k / nDCG@k dip when a cut lands mid-idea. Overlap is the cheapest lever to lift recall@k.

STRATEGY 2 · RESPECT MEANING

Semantic Chunking

WHAT IT IS

Split at logical boundaries — sentences, paragraphs, sections — instead of arbitrary lengths. Highly similar neighbours may be merged into coherent blocks.

HOW (LangChain)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
separators=['\n\n', '\n', '. ', ' ', ''], chunk_size=500
```



Best fit: well-structured, narrative or academic docs where continuity matters.



Pros

- Preserves the flow of ideas; keeps related concepts together
- Cleaner embeddings → better matches, higher retrieval accuracy



Cons

- More complex; yields variable chunk sizes
- Slightly higher compute than fixed-size



METRIC LEAN

Lifts MRR@k and nDCG@k — the answer chunk stays whole and ranks higher; recall@k improves as concepts stop fragmenting.

STRATEGY 3 · STRUCTURE-AWARE

Recursive Chunking

WHAT IT IS

Use a hierarchy of separators: split on high-level boundaries first, then recurse to finer ones only where chunks remain too big. Language-aware for code (def, class).

HOW (LangChain)

```
RecursiveCharacterTextSplitter.from_language(  
    language=Language.PYTHON, chunk_size=100, chunk_overlap=15)
```



Best fit: technical docs with clear structure — code repos, structured reports.



Pros

- More context-aware splits than fixed-size
- Block-based splitting is ideal for code and structured text



Cons

- More to configure than simpler methods
- Needs domain-specific separators for best results



METRIC LEAN

Strong nDCG@k on code/structured content — a function or clause stays intact, so the answer chunk is cohesive and ranks near the top.

STRATEGY 4 · SIZE BY COMPLEXITY

Adaptive Chunking

WHAT IT IS

Vary chunk size with text complexity: simple passages become larger chunks, dense/intricate passages become smaller ones — measured by lexical density or sentence length.

HOW (LangChain)

```
AdaptiveTextSplitter(min_chunk_size=300, max_chunk_size=1000,  
    complexity_measure='combined') # custom TextSplitter
```



Best fit: mixed-complexity docs — handbooks mixing simple intros and deep analysis.



Pros

- Spends small chunks where meaning is dense, large where it's sparse
- Reduces wasted tokens on simple sections



Cons

- Needs a complexity metric; harder to tune and debug
- More up-front computation



METRIC LEAN

Steadier recall@k across a varied corpus — fewer diluted embeddings in dense sections; modest MRR@k gains from tighter dense chunks.

STRATEGY 5 · MAKE CHUNKS SELF-CONTAINED

Context-Enriched Chunking

WHAT IT IS

Attach extra context to each chunk — a summary of neighbouring chunks or metadata — so a retrieved chunk carries enough background to stand on its own.

HOW (LangChain)

```
# window summary of neighbours prepended to each chunk
enriched = f'Context: {summary}\n\nContent: {chunk}'
```



Best fit: docs where sections interlock — multi-chapter reports, linked research.

✓ Pros

- Maintains coherence across parts of a document
- Boosts queries that span multiple segments

✗ Cons

- Higher storage and memory; extra preprocessing layer
- Can add repetition if not managed carefully



METRIC LEAN

Best recall@k for cross-segment questions — self-contained chunks stop the chunk-1/31 miss; nDCG@k rises, at index-size cost.

STRATEGY 6 · LET THE MODEL SPLIT

AI-Driven Dynamic Chunking

TECHADEMY

WHAT IT IS

An LLM reads the document and picks natural breakpoints so each chunk holds a complete idea, adjusting size on the fly to conceptual density.

HOW (LangChain)

```
# LLM prompted to return JSON array of idea-complete chunks
chunks = (chunking_prompt | llm).invoke({'document': doc})
```



Best fit: complex, multi-topic docs with a generous compute budget.



Pros

- Highly adaptive; captures semantic nuance
- Can significantly raise retrieval accuracy



Cons

- Depends on the LLM; expensive in compute / API calls
- Hard to standardise or reproduce exactly



METRIC LEAN

Top MRR@k and nDCG@k when idea-complete chunks match intent — but recall@k rides on LLM quality, and run-to-run variance is real.

ONE VIEW OF THE TRADE-OFFS

Six strategies, side by side

Strategy	Splits on	Chunk size	Cost / complexity	Metric lean	Best fit
Fixed-size	Fixed count	Uniform	Low	Baseline; overlap ↑recall@k	Uniform / simple text
Semantic	Logical breaks	Variable	Medium	↑MRR@k, ↑nDCG@k	Narrative / academic
Recursive	Separator hierarchy	Bounded	Medium	↑nDCG@k (code/struct.)	Code / structured docs
Adaptive	Text complexity	Varies by density	Med-High	Steadier recall@k	Mixed-complexity docs
Context-enriched	Base + neighbour ctx	Base + context	High	↑recall@k (cross-seg.)	Interlinked sections
AI-driven	LLM breakpoints	Idea-complete	Highest	Top MRR@k / nDCG@k	Complex, multi-topic

Rule of thumb: start simple, measure, then add complexity only where the @k numbers justify the cost. Hybrid setups (below) often beat any single row.

MATCHING THE SPLIT TO THE DOMAIN

Industry use cases



BFSI — Britannia Counties Bank

Semantic + rich metadata for policy/KYC docs; hybrid search + RRF to nail exact account/reg IDs.

Watch: *recall@k on compliance completeness*



Legal

Domain-specific + hierarchical chunking keeps clauses intact; never split a clause mid-obligation.

Watch: *nDCG@k — right clause ranked first*



Healthcare

Semantic + context-enriched on clinical notes; missing a contraindication is unacceptable.

Watch: *recall@k is the safety metric*



Software / code repos

Recursive, language-aware on def/class; ~100 tok / 15 overlap (MongoDB finding).

Watch: *precision + recall on code retrieval*



Customer support / KB

Fixed-size + overlap for speed; hybrid search; latency-sensitive at scale.

Watch: *MRR@k — fast first correct hit*



Research / long reports

Context-enriched / hierarchical for questions that span chapters.

Watch: *recall@k across sections*

FOUR QUESTIONS BEFORE YOU SPLIT

Choosing a chunking strategy

1 Document structure & type

- Structured (reports, articles) → semantic / recursive
- Code / technical → recursive, language-specific
- Mixed / unstructured → AI-driven or context-enriched

2 Query complexity

- Fact-based → smaller, direct chunks
- Analytical → larger, context-preserving chunks
- Multi-concept → keep related data together

3 Model constraints

- Mind LLM and embedding context windows
- Watch token usage to control cost
- Chunk must fit the embedding model's limit

4 Performance needs

- Latency-sensitive → lighter, simpler chunking
- Accuracy-critical → context-enriched / AI-driven
- Resource-limited → fixed-size or basic semantic

THE DIALS THAT ACTUALLY MOVE SCORES

Best-practice knobs

Size & overlap starting points

Content	Chunk size	Overlap
General text	200–500 tok	10–20%
Code / technical	100–200 tok	15–25%
Narrative	500–1000 tok	~12%



Overlap is your recall@k dial. More overlap = the same idea appears whole in more chunks = fewer misses — paid for in index size.



Baseline first — Fixed-size at several sizes; record metrics before adding complexity.



Hybrid per section — Text, tables and code each get the splitter that suits them.



Add metadata — Section, type, date on every chunk — enables filtering + context.



Preserve boundaries — Never cut mid-sentence; respect paragraph/section breaks.



Feedback loop — Refine from real queries and which chunks actually got retrieved.

WHERE CHUNKING IS HEADING

Advanced & emerging techniques



Domain-specific

Chunk to each domain's layout — legal clauses, medical sections, financial line items.



Multi-modal

Convert images/tables to text or summaries so non-text elements are retrievable too.



Query-aware

Adjust chunk size/selection at query time — small for facts, broad for exploration.



Neural chunking

Learned models predict optimal boundaries, balancing coherence and length.



Hierarchical

Multi-level chunks (section → subsection → paragraph) preserve document structure.

PROVE IT — DON'T GUESS

TECHADEMY

Measuring recall@k, MRR@k, nDCG@k

1

Build a gold set — For a set of real queries, label which chunk IDs are truly relevant. This is your ground truth.

2

Sweep k — Run each candidate strategy; compute recall@k, MRR@k, nDCG@k at k = 1, 3, 5, 10.

3

Read the knee — Pick the strategy + k where scores plateau — more k past the knee just adds noise and cost.

4

Fuse & re-rank — Layer hybrid search + RRF (from Anil's session) on the winning chunks to lift MRR@k / nDCG@k further.

5

Close the loop — Feed production queries + thumbs back in; re-label and re-run as the corpus drifts.

What good looks like

recall@k

High even at small k — the answer is in the first few chunks.

MRR@k

Near 1.0 — first relevant chunk sits at rank 1–2.

nDCG@k

Near 1.0 — strongly-relevant chunks are ordered above weak ones.

If two strategies tie on recall@k, prefer the higher MRR@k.

WHAT TO CARRY OUT OF THIS ROOM

Key takeaways



No universal strategy

Every method trades off — fixed to AI-driven. Experiment; there is no single winner.



Balance size & semantics

Big enough for context, small enough to stay efficient and un-diluted.



Preserve context

Cut on natural boundaries; add metadata and neighbour context for retrieval.



Measure, don't vibe

Grade every candidate on recall@k, MRR@k, nDCG@k against a gold set.



Hybrid excels

Mix strategies by content type; fuse vector + keyword with RRF.



Method over frozen values

Teach the dials; keep volatile sizes, k and model names in trainer notes.



The split is the foundation.

Choose it for your domain, prove it with recall@k , MRR@k and nDCG@k , and fuse the winners with hybrid search + RRF.



Reference: Debu Sinha — 'The Ultimate Guide to Chunking Strategies for RAG Applications' (Databricks Community). Metric layer & lab tie-ins: FORGE FDE Academy.